NLP Assignment 2

S. Moneer (2712847) , S. Mickus (2774595), R.R. Kanhai (2690999)

April 23rd, 2026

1 2.0: Group Agreement

1. Answer the VU group work Starting Questions on your own. To help you brainstorm goals in different roles, see the VU's website on group work. For question 9, generate a random question from here for the entire group and answer it. (2 points)

—– Everyone fill in the questions for themselves —–

Question 1: What are your weak points and strong points in group work?
R: Strength: Communicating my progress with the team. Helping others with understanding difficult concepts. Weakness: Properly planning my work.
Se: Strength: Clear planning and structuring, good at communication and collaboration. Weakness: Getting much done ahead of time. Especially by myself.
Si: Strength: Communication and collaboration, coming up with ideas. Weakness: getting ahead of the work and not letting it pile up until last minute.

Question 2: What do you hope to learn as a group during the project?
R: Get more understanding of the field NLP and how it is bigger than just LLMs.
Se: Practical programming experience with NLP
Si: More about NLP implementation, what to watch out for, and how to solve real world problems using NLP.

Question 3: What do you think the assignment is asking you to do?
R: Anything except training our own big model. Same as in the bachelor course Text Mining
Se: Complete some basic NLP tasks using simpler models that are probably not state of the art anymore.
Si: Use problems to show how things we learned in class can and should be used to solve actual problems.

Question 4: What role do you hope to fulfill during the group project?
R: I just want to be a good team mate and do my part properly.
Se: No particular role, just taking my part of the work and doing it well.
Si: Any role assigned/self-assigned to me.

Question 5: Which part of the group assignment do you want to take the lead on?
R: I like the last part. so 2.3
Se: The planning and structuring in the beginning.
Si: Assignment A2.1 and perhaps the bonus exercise.

Question 6: What are your goals for the project?
R: A decent grade mostly. Also getting more hands-on experience coding using NLP techniques.
Se: I think i can already study and work well, I want to start doing it with less stress.
Si: Try to understand the things taught and mentioned in class from a different perspective and how they could be used in different problems.

Question 7: What type of worker are you?
R: Dreamer rather than worker.
Se: I can work well and get a lot done in a short time, but I cannot consistently do this.
Si: I really don't know. Maybe someone who can get pulled into an interesting problem but struggles when the problem is easy and just takes a long time to complete.

Question 8: What personal development goal do you have for this group project?
R: I don't know. I would say trying to get some Python syntax in my head and do it without constantly looking for it using GenAI or Stack Overflow.
Se: Developing a more consistent work ethic.
Si: Develop a deeper understanding of the topics covered in this project and hopefully start to see how these topics could be applied to problems outside of the assignment as well.

Question 9: What management style do you prefer?
R: Are we talking about something like the waterfall method? Because I kind of like that one. But with limited time idk what to answer to this.
Se: Jokingly formal. A clear and somewhat rigid structure, riddled with jokes.
Si: I do not have a preference. I can work in any environment presented to me.

2. Share your answers in your first group meeting. Based on your discussion, draft a timeline of the work to be done in A2, how you will allocate the tasks amongst your group, and how you will communicate as a group, including when you will have group meetings. Each group member should sign this agreement

document (2 points)

We all had another course for which the deadline was on Sunday, 4 days before the deadline. We therefore agreed it was easiest to begin on Monday with an online team meeting. Here we decided to divide the tasks based on who had time when during the week. We agreed that Simonas would do 2.1 on Monday, Selem 2.2 on Tuesday and Rohan 2.3 on Wednesday and Thursday, with time left inbetween spent on the bonus task. Should there any difficulties, we will meet online or during the TA session.

3. Before you hand in your assignment, write a brief summary of how your group worked together and if you adhered to your contract. (1 point)

2 2.1: Text Processing & Zipf's Law

2.1 Definitions and Text Processing

"Word" was defined as any token containing at least one alphabetic character (using `[A-Za-z]`). This approach excludes pure punctuation and numbers while retaining hyphenated forms, preventing punctuation from being recognized as a "word".

A **Type** is a unique token string (case and form sensitive, including punctuation), while a **lemma** the different forms back into their base roots. The main problems when defining, were deciding whether to lowercase capitalization differences, handling acronyms (like U.S.), and dealing with some contradictions, where 2 different morphemes are treated as one.

2.2 Brown Corpus Analysis

The "News" genre shows longer sentences, rich in proper nouns and numerals, while the "Romance" genre uses shorter, sentences with a large number of pronouns. Both genres perfectly illustrate **Zipf's law**, which can be seen on the log-log plots, which show a linear frequency decay. The tail (that looks like a staircase) pattern presents itself because some words occur in-frequently or exactly once.

The graphs also show that in the "News" genre there is far more repetition with its top-ranked words occurring far more frequently as opposed to the "Romance" genre. This demonstrates a bias in the data.

2.3 Indian Corpus Pipeline

As for the Indian Language Corpus, we had to adapt out pipeline since NLTK's English tagger is meaningless for languages like Hindi and Telugu. We changed the corpus reader and relied on the native POS tags.

When the script was for the Telugu language, we found that our word classification filter rejected these scripts and showed that the language had 0 words in it. When working with the different language, we notices the languages like Telugu broke English lemmatizers. We believe this is due to the fact that suffixes were incorporated into the stems of the words. We also found that since standard tools assume English punctuation conventions, various native ”markers” were mishandled.

3 2.2: N-Gram Language Modeling

1. This is a basic n-gram model. Since the data provided is already space separated, the tokenizer simply splits it by whitespace. The independence of sentences is maintained by keeping them in separate lists, avoiding `(</s>, <s>)` bigrams for example. It then creates an occurrence count of n-grams along with their contexts (which is the words that preceded the last word. The model then calculates the probabilities, which also implements alpha smoothing. To generate another word, the prompt is treated as context and the probabilities of each other word to appear afterwards are calculated. From this probability distribution a new word is then sampled.

2. Alpha smoothing adds a baseline probability that any n-gram will appear, even if it was present in the data. This is especially useful in smaller datasets where there may be many combinations that do not appear. In practice this means a higher alpha value lowers the probability of the n-grams that already appear often, and raises the probability of n-grams that do not or barely appear. Specifically, the top 10 probability values become lower when alpha goes up.

3. Perplexity, as the name implies, calculates how ’surprised’ a language model is about a certain outcome. Basically, it calculates the difference between the outcome and the probability the model assigned to that outcome, so a perfect model would assign the correct answer a probability of 1, while a worse model would assign it a probability of, for example, 0.7. The perplexity metric also includes normalization for text length etc. In this task it could be implemented by feeding a text word for word into the model and ask it to predict the next word. You could then compute the perplexity based on the correctness of the answer and the probability distribution. However the text used should be of a similar context since the training data had a very limited vocabulary (820 words). The effect of alpha smoothing would be especially interesting in this case.

4. In natural language, the meaning of words can depend on context given much farther away in a sentence or even in a whole different sentence altogether. This means that to understand the semantical meaning of natural language, a large window is necessary. This is in contrast to the syntax of natural language, which can often be derived from a couple of adjacent words. This can explain why an n-gram model can produce sentences that look somewhat grammatically correct while not conveying any actual meaning and why an LLM can produce coherent texts that seem to convey actual meaning.

4 2.3: Word Vectors

1. Representing words as vectors is helpful because it mathematically captures the distributional hypothesis—that words occurring in similar contexts tend to have similar meanings. By mapping words to continuous multidimensional points in geometric space, models can compute relationships by looking at proximity. However, these vectors are often not helpful when determining the meaning of polysemous words or homonyms. Because a word is assigned only a single vector, that vector acts as a weighted average of all its contexts. For example, in our GloVe testing, words like "apple" and "bank" failed to show multiple meanings in their top-10 similar words, as their dominant contexts completely overshadowed the secondary meanings. Only "mouse" successfully showed multiple meanings, including computer devices (trackball, joystick) and its animal/cartoon association (mickey).

2. We implemented two methods for representing words: a count-based co-occurrence matrix with dimensionality reduction (SVD), and prediction-based pre-trained GloVe embeddings. When comparing the 2D plots of our chosen words (e.g., "political", "candidates", "Mayor" vs. unrelated words like "coolest", "shot"), it stood out that the count-based method only clustered words based on direct adjacency in our limited corpus subset. In contrast, the prediction-based GloVe vectors clustered the semantic groups much more tightly and accurately, as they draw from massive datasets. Compared to how humans represent words, both methods are severely limited. Humans use rich sensory, emotional, and cultural associations to understand meaning, and effortlessly switch between contexts, whereas these models blindly rely on statistical co-occurrence.

3. The simple tests in the code are a helpful starting point, but they don't fully capture a word's real meaning. For example, our cosine distance test showed that an opposite word ("bad") was actually closer to "good" than a synonym ("fantastic"). This happens because the model only checks if words are used in the same context, and opposites often fit into the exact same sentence structures. Additionally, the analogy test ("man : profession :: woman : x") showed strong gender biases. The model associated men with jobs like "boss" or "mechanic," and women with "nurse" or "receptionist." This proves that these evaluations mostly reflect the statistics and biases of the texts they were trained on, rather than understanding the actual, objective meaning of the words

5 Bonus

5.1 Validity of Independence Assumption

Our results show that the independence assumption is false.

For High PMI examples (strong positive dependence):

- "per cent" - These two words almost always appear together. "per" almost never precedes anything else, and "cent" similarly almost never follows anything else. The unigram model, which treats them independently, underestimates their joint probability.

- "United States" - "United" by itself could follow anything, but in a news/general corpus it almost always means the country.

For Low PMI score examples (strong negative dependence):

- "the the" - "the" is almost never immediately followed by another "the". Because both unigrams are extremely frequent (high $C(w)$): $C(w_1) \cdot C(w_2)$ is huge, but $C(\texttt{"the the"})$ is tiny, producing a PMI which is very negative.
- "of of" - Same argument for another very high-frequency word combination. The independence assumption predicts "of of" occurring as often as "of the", scaled by the relative frequency of the words "of" vs "the", which is wrong.

We find that independence fails for both "high" and "low" PMI scores and these examples illustrate that any evaluation of a unigram will be penalized heavily by both extremes, thus falsifying the independence assumption.

5.2 PPMI vs PMI

The top-20 PPMI list is identical to the top-20 PMI list because PPMI only affects negative values. The highest PMI pairs are all strongly positive and therefore unchanged. The difference appears at the bottom for all pairs with negative PMI (e.g. "the the", "of of"), which receive PPMI of 0, making them indistinguishable from pairs that were never observed together.